

```
# -*- coding: utf-8 -*-
```

```
"""
```

g729_annex_scan.py - G.729 구현체 Annex 식별 보조 스크립트 (IDA 7.x ~ 9.x, IDAPython)

사용: IDA에서 File > Script file... 로 실행. Output 창에 히트 주소와 추정 결과가 출력된다.

주의: 바이트 패턴은 ITU-T G.729(01/2007) 문서의 수치에서 유도한 값이며, 판정은 '후보'일 뿐이다.

최종 판정은 함수 구조 비교 및 테스트 벡터 bit-exact 검증으로 확정할 것.

```
"""
```

```
import re, struct
```

```
import idautils, ida_bytes, ida_segment, ida_name
```

```
def p16(fmt, *v): return b"".join(struct.pack(fmt + "h", x) for x in v)
```

```
def p32(fmt, *v): return b"".join(struct.pack(fmt + "i", x) for x in v)
```

```
# (그룹, 설명, 값목록, 폭)
```

```
SIGS = [
```

```
    ("COMMON", "전처리 HPF 140Hz b[] (Q12)", (1899, -3798, 1899), 16),
    ("COMMON", "전처리 HPF 140Hz a[] (Q12)", (4096, 7807, -3733), 16),
    ("COMMON", "후처리 HPF 100Hz b[] (Q13)", (7699, -15398, 7699), 16),
    ("COMMON", "후처리 HPF 100Hz a[] (Q13)", (8192, 15836, -7667), 16),
    ("RATE8K", "8 kbit/s 비트할당 bitsno (합 80)", (8, 10, 8, 1, 13, 4, 7, 5, 13,
    ("MAIN", "grid[61] 선두 (본체 LP->LSP)", (32760, 32723), 16),
    ("ANNEX_A", "grid[51] 선두 (Annex A LP->LSP)", (32760, 32703), 16),
    ("ANNEX_B", "SID 비트할당 {1,5,4,5} (합 15)", (1, 5, 4, 5), 16),
    ("ANNEX_B", "VAD 결정경계 a1..a4 (Table B.1)", (23488, -30504, -32768, 26214),
    ("ANNEX_B", "VAD 결정경계 b1..b4 (Table B.1)", (28521, 19446, -32768, -19661),
    ("ANNEX_B", "VAD 임계값 T1,T2 (Table B.1)", (671088640, 738197504), 32),
    ("ANNEX_D", "6.4k ACELP 트랙0 (Table D.2)", (1, 3, 6, 8, 11, 13, 16, 18),
    ("ANNEX_D", "6.4k ACELP 트랙1 (Table D.2)", (0, 1, 2, 4, 5, 6, 7, 9, 10,
```

```
]
```

```
# 심볼이 남아 있는 경우(.ko/.so/디버그 빌드) 이름으로 바로 판별
```

```
NAME_HINTS = [
```

```
    ("MAIN", r"^(Coder_ld8k|Decod_ld8k|Pitch_ol|Pitch_fr3|ACELP_Codebook|Post)
    ("ANNEX_A", r"^(Coder_ld8a|Decod_ld8a|Pitch_ol_fast|Pitch_fr3_fast|ACELP_Code
    ("ANNEX_B", r"^(?i)^(vad|init_cod_cng|cod_cng|dec_cng|qua_sidgain|lsfq_noise|c
    ("ANNEX_D", r"^(?i)(ld8kd|_g6k|phdisp|6k)",
    ("ANNEX_E", r"^(?i)(ld8e|bwfw|track_pi|acelp_e)",
    ("INTEG", r"^(?i)(ld8cp|ld8f|ld8g|ld8h|ld8i|mus_dtct|musdetect)",
```

```
]
```

```
def scan(pattern):
```

```
    hits = []
```

```
    for s in idautils.Segments():
```

```
        seg = ida_segment.getseg(s)
```

```

        data = ida_bytes.get_bytes(seg.start_ea, seg.end_ea - seg.start_ea) or b"
        i = data.find(pattern)
        while i != -1:
            hits.append(seg.start_ea + i)
            i = data.find(pattern, i + 1)
        return hits

found = {}
print("=" * 70)
print(" G.729 Annex 식별 스캔")
print("=" * 70)
for grp, desc, vals, width in SIGS:
    for endian, tag in ((" < ", "LE"), (" > ", "BE")):
        pat = p16(endian, *vals) if width == 16 else p32(endian, *vals)
        hits = scan(pat)
        if hits:
            found.setdefault(grp, []).append(desc)
            print("[%8s] %-36s %s x%d first=0x%X" % (grp, desc, tag, len(hits),

print("-" * 70)
for ea, name in idutils.Names():
    for grp, rx in NAME_HINTS:
        if re.search(rx, name):
            found.setdefault(grp + "(sym)", []).append(name)
            print("[%8s] symbol %s @0x%X" % (grp, name, ea))

print("-" * 70)
g = lambda k: k in found or (k + "(sym)") in found
if not g("COMMON") and not g("RATE8K"):
    print("G.729 고정소수점 흔적이 약함 -> 부동소수점(Annex C/C+) 또는 비표준 구현 가능성 확인")
core = "Annex A(저복잡도)" if g("ANNEX_A") and not g("MAIN") else \
    "본체(Full)" if g("MAIN") and not g("ANNEX_A") else \
    "본체+Annex A 모두 포함(또는 판별 불가)"
print("8k 코어 추정 :", core)
print("Annex B(VAD/DTX/CNG):", "있음" if g("ANNEX_B") else "흔적 없음")
print("Annex D(6.4k)          :", "있음" if g("ANNEX_D") else "흔적 없음")
print("Annex E(11.8k)         :", "있음(심볼)" if g("ANNEX_E") else "바이트 시그니처 없음")
print("통합형(F/G/H/I/C+)    :", "심볼 흔적 있음" if g("INTEG") else "심볼 흔적 없음")
print("* 최종 판정은 보고서 6.8절 판정표 + 6.10절 bit-exact 검증으로 확정")

```